PROJECT CASE STUDY

Restaurant & E-Commerce Core API

Arquitectura Base Empresarial (Template Repository)

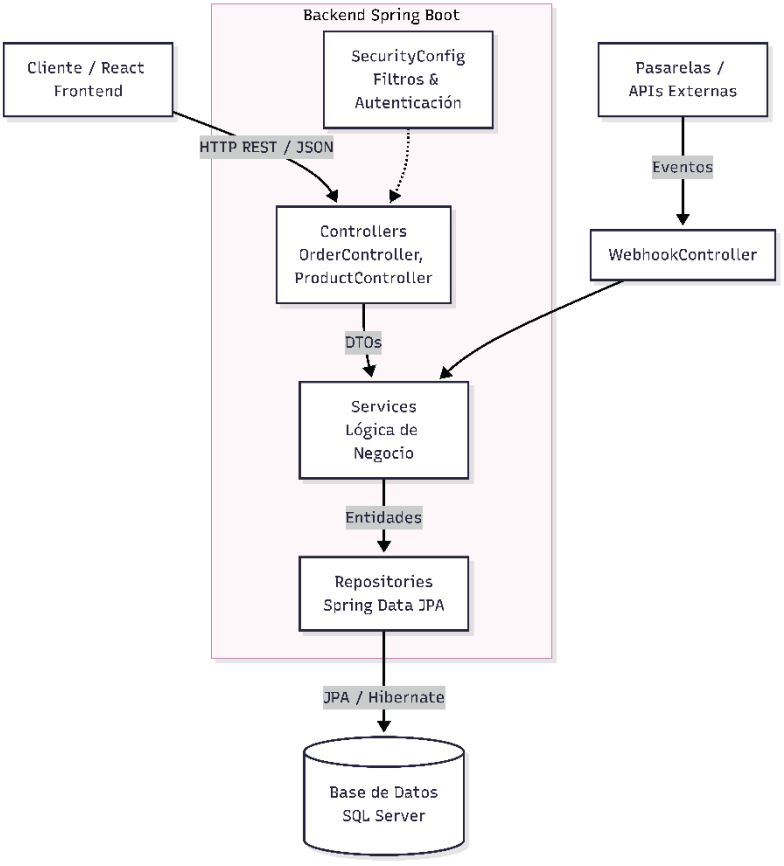
Resumen del Proyecto

Este proyecto consiste en el desarrollo de la arquitectura base del backend para los productos de la agencia. El objetivo fue diseñar un núcleo API REST estandarizado, escalable y seguro en **Java Spring Boot** que permita reducir el *time-to-market* al implementar nuevos sistemas de menús digitales, gestión de pedidos y comercio electrónico para futuros clientes.

Java 21 | Spring Boot 3.x | Spring Data JPA | SQL Server | Maven | REST API

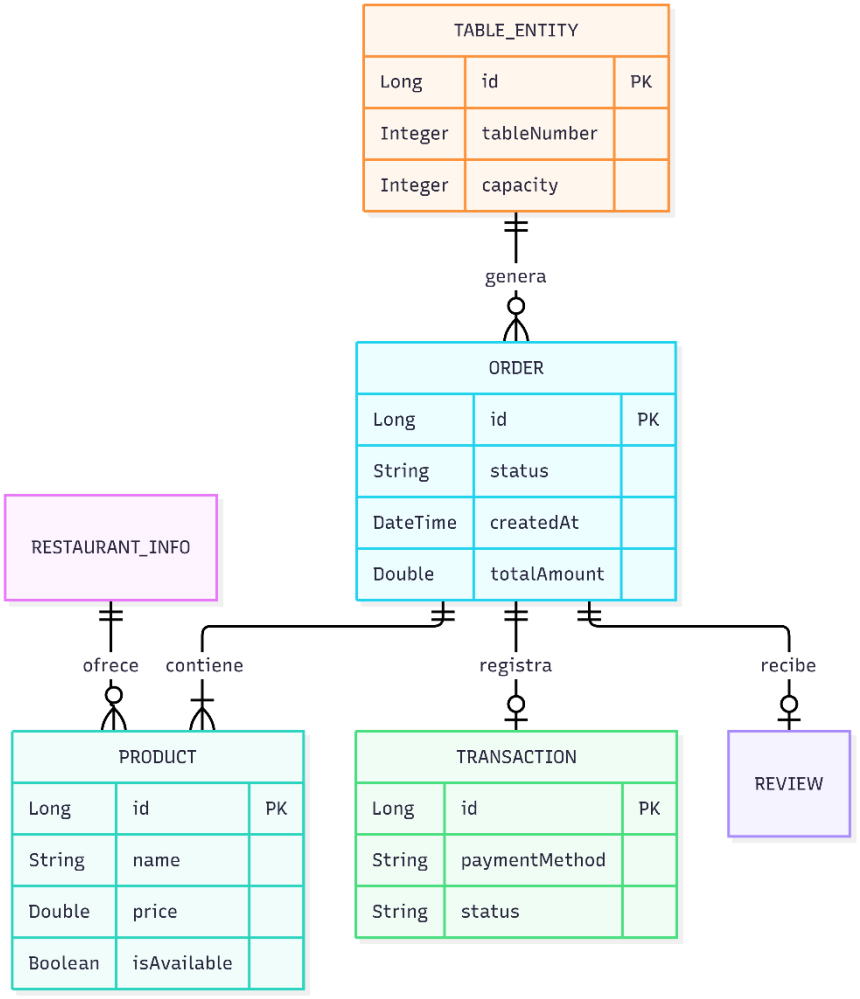
Arquitectura Multicapa (Clean Code)

El sistema se construyó bajo una separación estricta de responsabilidades (SoC) utilizando el patrón de diseño MVC adaptado para APIs. Esto garantiza un código fácil de mantener, testear e iterar por múltiples desarrolladores.



Modelo de Dominio (Entidades Clave)

El diseño de la base de datos relacional está optimizado para flujos de restauración, mapeado íntegramente mediante anotaciones de Hibernate (ORM).

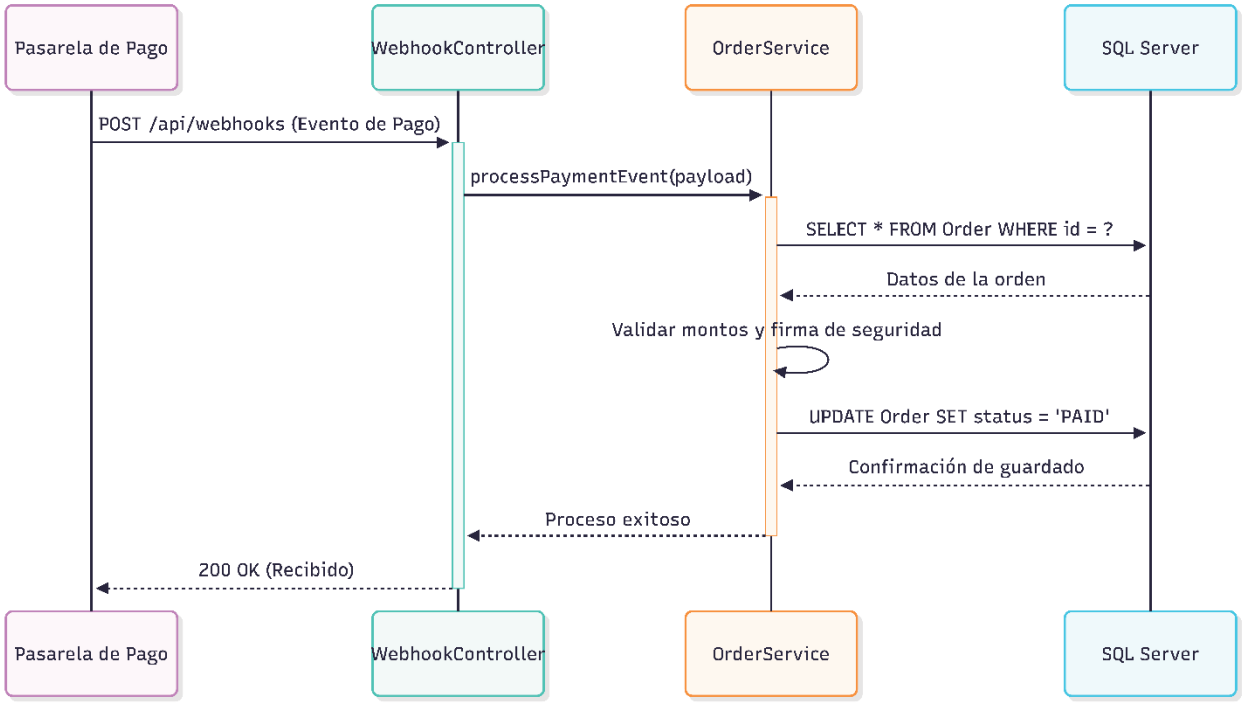


Estructura Base de la APIREST

Método	Endpoint	Propósito y Lógica
`GET / POST`	**/api/products**	Consulta de catálogo para clientes y gestión de inventario para administradores.
`POST`	**/api/orders**	Recepción de órdenes entrantes, validación de stock y creación del ciclo de vida.
`GET`	**/api/restaurant-info**	Sincronización inicial del frontend con las configuraciones del comercio.
`GET`	**/api/reports**	Agregación de datos financieros y operativos (Ventas, órdenes completadas).
`POST`	**/api/webhooks**	Recepción segura de eventos externos (ej. confirmaciones asíncronas de pasarelas de pago).

Flujo de Integración (Webhooks)

Para garantizar la fiabilidad del proceso asíncronos (como la confirmación de pago), se implementó un controlador de eventos que procesa las notificaciones de sistemas externos de forma segura.



Seguridad y Buenas Prácticas Implementadas

La arquitectura base no solo define rutas y bases de datos, sino que establece los estándares de calidad del equipo de desarrollo:

- **Manejo Global de Excepciones:** Un controlador centinela intercepta los errores del sistema y los formatea en respuestas HTTP estandarizadas, ocultando trazas de error sensibles al usuario final.
- **Aislamiento de Entorno:** Las credenciales de base de datos y configuraciones sensibles se inyectan a través de variables de entorno (`application.properties`).
- **Seguridad Centralizada:** Integración de la clase `SecurityConfig` preparada para extenderse con validación JWT e interceptores CORS.

Roadmap y Próximos Pasos (Fase 2)

El desarrollo de este *Core* es iterativo. Las próximas etapas de maduración de la arquitectura incluyen:

- **Integración de Spring Security:** Implementación de autenticación basada en tokens JWT para diferenciar roles (Admin, Cajero, Mesero, Cliente).
- **Documentación Automática:** Configuración de Swagger (OpenAPI) para que los desarrolladores frontend puedan visualizar y probar los endpoints interactivamente.
- **Testing Unitario:** Incorporación de pruebas automatizadas con JUnit 5 y Mockito en la capa de servicios para asegurar la fiabilidad de las reglas de negocio.